

Nine systems and why they exist

Each of these was built to solve a problem I actually had. This is the record of why each one was started, what it was supposed to do, the design decisions that turned out to matter, and where it stands now — written for someone deciding whether any of it belongs in a real business.

~181,000	15 months	One operator	No team
LINES OF ORIGINAL CODE	MAY 2025 → TODAY	ONE SERVER	NO FUNDING

WHO BUILT THIS

I am an expat from Russia, living in Panama, with an executive background — seven years as Deputy CEO and Chief Legal Officer in e-government. No team, no funding, no manual coding background.

Those facts set every constraint in this document: one always-free server, a phone-first control room, and a five-provider model chain so that no single account running dry can stop the work.

So the honest claim is narrow and I will not stretch it: **these are working systems that solve real problems and have never been put in front of anyone who needed them.** What follows is engineering provenance — why each was built and how it was designed — not a performance record. Every number below was read from the running systems today.

THE LAB IN ONE PARAGRAPH

Aldeazz is a live AI Lab built via an AI-augmented workflow in Claude Code and Cursor, running on one Oracle Cloud server: agents that research, sell, publish, tutor, and keep the lights on. **The control room is Telegram. The ledger is HubSpot.** The factory is TypeScript and Python with a five-provider LLM chain — Claude → Groq → OpenAI → Gemini → Grok — so if the primary model goes dark, the Lab does not.

A LangGraph job engine discovers and scores roles with a human on the last click. EspaLuz teaches Spanish on WhatsApp and Telegram with memory, voice, emotional intelligence and payments. Atlas reads public ad libraries and flags which creative angle is opening before it saturates. A GEO/AEO/Tech-SEO engine keeps the Lab citable — portfolio, free visibility audit, daily blog, AI Ops Wiki, eleven-episode podcast — each with canonical URLs and UTM/Atlas IDs so a mention can be traced.

One server that never sleeps, doing the work of a small company: it finds potential customers, researches them, writes to them, follows up, keeps the CRM tidy, publishes an article every day, watches the ad market for openings, finds and applies to jobs, and sends me a briefing every morning at 8am.

HOW THIS WAS BUILT

AI-augmented, and that is the point

I am not a computer-science graduate, and I did not type most of this code. I direct AI to write it — Claude Code and Cursor — and I have since the first commit. On several of these repositories the majority of commits carry my AI agent's authorship rather than mine. I state that plainly because anyone can see it in a `git log` in ten minutes, and because I think it is the most interesting thing about the work rather than the most embarrassing.

What it changes is where the effort goes. My job is not producing the implementation. It is deciding what to build, designing how the pieces fit, and — the part that consumes the most hours — **verifying that what came back is actually true.**

WHY THE DISCIPLINES IN PART 3 EXIST

Every one of them was earned because AI-generated code is confidently wrong in specific, repeatable ways. The rules are not general engineering wisdom I read somewhere; they are scar tissue from this exact way of working:

It reports success it did not achieve — so I measure at the edge, never at the source. A publisher told me an article was live while it existed nowhere.

It writes a second implementation rather than finding the first — so: one file, one writer. Two processes writing one sitemap quietly destroyed 61 URLs on every run.

It produces plausible numbers — so the daily article generator collects production evidence *before* it writes and refuses to publish any figure it cannot trace to a measurement.

This is the honest trade, and I would rather state it than have it discovered. **A whiteboard interview measures the skill this toolchain has made abundant.** What stayed scarce is knowing which output to distrust, and being able to prove the difference. I have one kind of evidence for that: eight systems that have run unattended on one server for fifteen months, the longest single process up 27 days without a restart, and a public record of every time one of them fooled me.

I would rather be judged on a system that survived a provider outage at 3am than on whether I can invert a binary tree on a whiteboard. One of those two things has happened to me in production.

The shared spine

Every product inherits the same infrastructure, and that was a deliberate early decision rather than an accident of laziness. Building eight separate stacks alone is not possible; building one substrate and eight applications on it is.

LAYER	CHOICE	WHY THIS AND NOT THE OBVIOUS ALTERNATIVE
Compute	One Oracle Cloud server	Oracle's always-free tier, plus a one-year startup credit allocation from Oracle . Building alone without funding means infrastructure has to cost nothing — and a single box makes failures observable in one place.
Control	Telegram, not a web dashboard	I operate from a phone most of the day. A dashboard needs a laptop and a login; a chat app is already open. Approvals happen in one tap where I actually am.
Ledger	HubSpot CRM	Every agent writes to one place with a prefix naming its author, so provenance is a filter rather than an investigation.
Models	Five providers in a ranked chain	A single API key is a single point of failure for the whole business. Three chains, ordered per use case — quality leads with Claude, classification and bulk lead with cheaper models that are good enough.
Language	TypeScript and Python	TypeScript for anything touching money or CRM state, where the type checker catches a whole class of mistake before deploy. Python where the ML ecosystem lives.

THE DECISION I WOULD DEFEND HARDEST

The five-provider chain. On 19 August my Anthropic balance hit zero. Nothing stopped: the chain fell through to OpenAI in 2.4 seconds and the system kept answering leads. The design cost about a day; it has since absorbed a total provider outage, a rate limit, and a model deprecation without a single lost request.

The part that took longer to learn: the order has to differ by use case. Reasoning models return empty strings under low token budgets, so a classification chain that leads with one fails silently — worse than erroring. Client-facing copy leads with the strongest model; classification leads with the cheapest that clears the bar.

Every product, and the need behind it

Each entry carries its repository, its public face, and the CRM prefix its records are written under — so any claim here can be opened and checked. **None of these started as a product idea.** Each one started as something I needed to survive a specific week in Panama.

EspaLuzFamilybot · EspaLuzWhatsApp · Python

EspaLuz — a bilingual expat advisor for arriving families

The first thing I built, because four of us arrived at once and none of us had the language

LOC **52,487** COMMITS **595** SINCE **May 2025** MEMORY **LangChain + pgvector RAG** DESIGNED FOR **21 countries**

repo: [EspaLuzFamilybot](#)

repo: [EspaLuzWhatsApp](#)

live: [EspaLuz on Telegram](#)

live: [EspaLuz on WhatsApp](#)

CRM: [\[ESPALUZ\]](#) · 69 deals

WHY I BUILT IT

Survival, and not only mine. Three generations arrived in Panama at once — my daughter, my retired American father, my retired Russian mother, and me. A single mother, and the only one of the four who could solve any of it. Four people, four different problems, one language none of us had. A child starting at a new school. Two retirees facing residency paperwork, clinics, pharmacy counters and banks.

Language apps teach vocabulary on a schedule, and none of that was the problem. The problem was **the sentence in front of us, right now** — and the thing nobody tells you: which form, which queue, what you are actually agreeing to. Every expat family arriving anywhere has that same first month.

DESIGNED

Two front ends over shared logic: in Latin America WhatsApp is the internet, Telegram is where early adopters are. Persistent per-family memory, voice in and voice out, image understanding for the forms, payments wired for trials.

The constraint that shaped everything: **emotional intelligence before correctness**. A parent asking how to tell a teacher their child is frightened does not need a grammar drill. Neither does a seventy-year-old at a pharmacy counter.

Two layers of memory, because one was not enough. A conversation needs recent context. A family arriving needs something else entirely — the answer in November depends on the form they filled in June. So short-term context sits in a LangChain conversation layer, while everything durable goes into a **pgvector store on Oracle Autonomous DB and is retrieved by relevance rather than recency (RAG)**.

That distinction is what turned a tutor into an advisor. "*Which queue did I use last time?*" and "*what did the clinic say I needed?*" are not language

questions — they are retrieval questions over a family's own accumulated history, and nobody else can answer them. Vocabulary fits in a prompt; fifteen months of one household's arrival does not.

Then it outgrew tutoring. What the four of us actually needed was not translation but lived practicality — how residency really works, what to say at the counter, how a lease conversation goes, which queue is the right queue. So EspaLuz became an **expat advisor that teaches the language while it answers**. Memory is per family rather than per user, because the second question always depends on the first, and because a household arrives together.

RUNNING

Live on both platforms with real families using it, at the scale you would expect from a product that has never had an advertising budget, a launch, or a community behind it. **The engineering is finished; the distribution has never been attempted.**

Nothing in it is Panama-specific by design — the same arrival, the same paperwork and the same first month repeat across **21 Spanish-speaking countries**.

WOULD FIT

Relocation and residency firms, retirement-abroad services, international schools, and immigration practices — all of whom are already talking to arriving households on WhatsApp. Panama alone is a major destination for North American retirees, and my own family is the proof that the buyer is rarely the learner: **the adult child arranges it for the parents**. The audience exists and is assembled; I have never had access to it.

VibeJobHunterAIPA_AIMCF · Python · LangGraph

VibeJobHunter — job discovery with a human on the last click

Built to find work with skills I had just taught myself

LOC 33,988 COMMITS 543 SOURCES 11 EVALS 131 tests / 4 layers

repo: VibeJobHunter

CRM: [HIRING-VJH-LEAD] · 1,166

CRM: [HIRING-VJH-SERP-LEAD] · 925

WHY I BUILT IT

I needed a job, in a field I had entered sideways. Seven years as a Deputy CEO and Chief Legal Officer, then a deliberate move into AI — with no CS degree and no manual coding background. Applying by hand meant a hundred hours of repetitive work while competing against people whose CVs read conventionally. I automated the part that was mechanical so I could spend my hours on the part that was not.

DESIGNED

A LangGraph state machine — gate, score, route, notify — with SQLite checkpointing so a crash resumes rather than restarts, and a deliberate human-approval interrupt.

The decision I made against myself: it originally auto-applied. An audit showed auto-apply produced volume and zero outcomes — theatre that felt like progress. I removed it. Fewer applications, honest numbers, and the scoring model finally had clean feedback.

Scoring is done by an LLM judge, because keyword matching cannot tell an ML-research role from an AI-automation one. But a judge nobody audits will drift, so it has two counterweights.

An eval harness — 131 tests across four layers. The first two layers run with the AI disabled and no network, so they are free and fast enough to run before every edit to a scoring rule. Golden-set routing is tested deterministically, which is the interesting part: proving the predictable half of a non-deterministic system still behaves. One test class is dedicated to **bias compensation**, because the failure that matters is not a crash — it is a judge that quietly starts preferring the wrong thing.

And a feedback loop. It reads what I actually did in the CRM — which roles I acted on, which I rejected — and writes that back into the judge's prompt. It learns my *reasons*, not only my verdicts, including the explanations I type and the screenshots I attach.

The incident that shaped it: deterministic keyword bonuses were silently outvoting the model, so a role could score well for containing the right words after the judge had correctly said no. The fix was a trust clamp bounding how far the keyword layer can override the model, plus fingerprint deduplication. It also **fails closed on an LLM outage** — no judgment is better than a fabricated one.

RUNNING

11 sources hourly, 1,166 roles surfaced, learning from the reasons I type when I reject something. **It found the interview I sat this week.**

WOULD FIT

Recruiting firms and talent marketplaces. The transferable shape is *scored routing with a human interrupt and a feedback loop from real outcomes* — the same as underwriting, triage or moderation.

AIPA_AITCF · TypeScript

CTO AIPA — the operations brain

Built to find clients, and to keep everything else alive

LOC 60,132 COMMITS 977 DOCS 164 files COMMANDS 82

repo: AIPA_AITCF

live: Ops SOP

CRM: [CLIENT-CTO-INQUIRY] · 44

CRM: [CLIENT-CTO-SERP] · 261

CRM: [CLIENT-MANUAL] · 116

WHY I BUILT IT

I needed customers and had no sales team, no network in Panama, and no budget. So I built the sales operation as software: find businesses that match, research them, write to them, follow up, and keep the ledger honest. The second need was quieter — with eight systems running, I could not tell each morning whether anything had broken overnight. This is the answer to both.

DESIGNED

Telegram as the control plane, HubSpot as the ledger, a webhook API as the front door for every other agent. The lead concierge is the piece I would show first: a form submission becomes a CRM record, a drafted reply, and a one-tap approval card — then delivery and open events write back to the deal.

The other half is the technical co-founder I did not have. The repo is named AITCF for a reason: the same Telegram bot is a coding assistant. `/code` and `/fix` generate a change and open a real pull request through the GitHub API; `/diff` shows the before and after; `/apply` and `/approve` keep a human on the last click; `/batch` and `/multifile` carry one edit across several files in a single commit. Around the typing sit the parts that are actually co-founder work — `/architecture` to explain the shape of a repo, `/decision` to record why an architectural choice was made, `/debt` to track what was knowingly deferred, and `/explaincode`, `/gentest`, `/error` and `/logs` to work a failure back to its cause. That is what makes a phone-first control room more than a status screen: a fix can be reviewed, approved and shipped from a queue.

The non-obvious decision is **precedence**. Two systems can write the reply. Redundancy alone means the faster one always wins, so the better one never runs. A cached health verdict decides who gets first refusal, with a five-minute grace and a twenty-minute safety net.

RUNNING

82 commands. Briefing at 08:00. Autonomous prospect research in ~90 seconds. A synthetic lead pushed through the real pipeline daily at 07:45 — silent on pass, alerting on failure.

WOULD FIT

Any small team drowning in inbound where a human must approve the reply — agencies, clinics, brokerages. The approval card is the product.

aideazz · TypeScript

The visibility engine — and my own audit API

Built to prove my own professional face is findable, not to assume it

SITEMAP 137 URLs INDEXED 96 AUDIT 34 checks

repo: aideazz

live: free audit

live: llms.txt

live: GEO manifest

live: portfolio

WHY I BUILT IT

I had no professional face in a new country and no way to know whether the one I was building was visible. Buyers and recruiters increasingly ask an assistant before a search engine. So I made everything machine-readable for answer engines — and then built the measurement, because a claim I cannot check is worth nothing. The free audit exists because I needed to run it on myself first.

DESIGNED

Content prerendered to static pages and emitted through a sitemap, `llms.txt`, a GEO manifest, JSON-LD and a citation file. A weekly probe asks real answer engines whether they quote the page, and that result feeds back into what gets written.

The rule enforced hardest: **the tracker exits non-zero when it cannot measure**, so a silent zero can never be mistaken for a measured zero.

RUNNING

137 URLs, 96 pages indexed, all seven machine-readable surfaces returning 200. Honest caveat: my own citation rate measured **0%**. The engine reports the truth including when it is unflattering.

WOULD FIT

Any agency selling SEO without an answer-engine story, and any business whose customers research before buying. The audit is the door-opener — a specific, checkable report in a minute.

aideazz · AIPA_AITCF · EspaLuz_Influencer

The publishing engine — blog, podcast, social

Built so my work has a public face without me performing on social media daily

ESSAYS 122 PODCAST 11 episodes CADENCE Daily 14:30

live: blog

live: podcast

live: Dev.to

live: X

live: LinkedIn

repo: EspaLuz_Influencer

CRM: [ESPALUZ-ORG] · 10

WHY I BUILT IT

I needed to be visible to strangers while having no hours to spend being visible. One person building alone cannot also post to LinkedIn, Instagram, X and a blog every day. So the Lab publishes on my behalf — but only things that actually happened, which is the constraint that makes it defensible rather than noise.

DESIGNED

Daily article at 14:30, cross-posted with canonical URLs so the original always wins. Bilingual social atoms routed out through scheduled posting. Every link carries UTM and Atlas IDs so a mention can be traced back to a source.

Grounded generation: the daily article collects production evidence *before* it writes — process uptimes, real commit messages, CRM counts — and refuses to publish if a number cannot be traced to a measurement.

RUNNING

122 essays, 11 podcast episodes, publishing daily and unattended. Every article carries the command that produced its evidence.

WOULD FIT

Any founder-led business that needs authority content and has no time to write it. The rare part is not generation — it is the refusal to publish an unverifiable claim.

aideazz · generated, never hand-edited

AI Ops Wiki — the learning record

Built to prove I am learning an engineering discipline, not vibe-coding

INCIDENTS **9** NAMED FAILURE MODES **9** UPDATED **Per session**

live: AI Ops Wiki

repo: aideazz

WHY I BUILT IT

This is the one I would point at if someone doubts the AI-augmented route produces real engineers. Every working session that produces a transferable lesson writes one entry: what broke, why, what the failure mode is called, and the verified number that proves it. It is a public, dated record of me learning in production — including the times a system fooled me. Nobody keeps this unless they are actually paying attention.

DESIGNED

Entries are structured data with a required verification field; the page is generated from them and never hand-edited, so the HTML cannot drift from the record. Scrubbed of client data before publishing. Concepts are cross-linked, so a new incident inherits the vocabulary of the old ones.

RUNNING

Nine incidents, each with a named failure mode — *redundancy is not precedence, acknowledgement is not completion, the wall was on the page, not the data*. Written the same day the thing broke.

WOULD FIT

Any engineering team that wants a postmortem habit and has never made one stick. Also the fastest way for a stranger to judge how I think.

inside AIPA_AITCF · atuona.xyz

Atuona — an automated AI film studio

Source clips in, finished film out — nobody touches an edit timeline

FILMS **6** VIDEO PROVIDERS **Multi, with fallback** EDIT STAGE **Fully automated**

live: Atuona

live: AI Film Studio

repo: AIPA_AITCF

WHY I BUILT IT

AI video is becoming a real production category, and almost every tool stops at generating a clip. Everything that turns clips into a *film* — pacing, voice timing, transitions, text, mixing, title cards — is still done by hand in an editor. I wanted to know whether that second half could be automated end to end.

The source material is my own work. ATUONA is an underground aesthetic gallery I designed and built myself — multidisciplinary NFT art: written work, imagery and film, presented as claimable drops with a wallet-connect interface and the film studio built into it. Using it as the corpus was a practical decision as well as a personal one — I needed material I owned outright, so nothing I published carried a licensing question. The films are real output, not a demo reel — they are how the pipeline gets tested against work I actually care about.

DESIGNED

Multi-provider video generation with fallback, because video APIs fail more often than text ones and each fails differently. Everything downstream of generation is a single script per film: change the constants at the top, run, publish.

The automated edit is the actual work. Voiceover is synthesised and then *locked to its clip* by a duration formula rather than sequenced on its own timeline — sequenced independently, narration drifts out of sync within three cuts. Short clips are stretched with slow-motion, never a held frame, because a freeze catches an actor mid-blink or mid-word. Transitions crossfade video and audio together on a running-offset accumulator so the timeline stays exact across a dozen joins. On-screen text fades out over each clip's final second, so a line can never straddle a dissolve.

The audio is mixed, not merely added. Music sits under a voice bus with **sidechain ducking**, so the score drops automatically whenever narration speaks and returns when it stops — then the whole mix is normalised to **−16 LUFS**, the broadcast loudness target, so films sound consistent against each other and against everything else a viewer plays.

The detail that proves it shipped: the intro card must not fade in from black. The gallery's video element uses frame 0 as its poster, so a fade-in makes the player look broken before anyone presses play. Every render is verified by extracting frame 1 and confirming the title card is there. That is not a thing you know before publishing — it is a thing you learn from a gallery that looked dead.

RUNNING

Six films live in the public gallery, each with its own score, narration, generated poster and title cards. The full pipeline — every ffmpeg chain, every setting and the reason behind it — is written up as a compilation guide in the repo, including the errors already paid for.

WOULD FIT

Brands, agencies and content studios producing AI video at volume — where the bottleneck is never generating a clip, it is the human hours between clips and a publishable film. Two parts transfer directly: multi-provider fallback so one outage cannot kill a campaign, and the automated edit, mix and loudness stage that removes the editor from the loop entirely.

inside AIPA_AITCF · seven verticals

Atlas — radar for advertising angles

Built to give me something to sell that was worth a stranger reading

VERTICALS 7 CADENCE Weekly LOOP Detect → sell → measure

live: Atlas

repo: AIPA_AITCF

CRM: [CLIENT-ATLAS] · 35

WHY I BUILT IT

Cold outreach with nothing to say fails, and I had nothing to say. Public ad libraries are open data almost nobody reads systematically. If I could tell a business which creative angle was opening in their market before it saturated, I would have a reason to write to them that was about them rather than about me.

DESIGNED

Detect, not predict. That constraint is in the design notes and it is the whole discipline: the system reports what is observably running and for how long, and refuses to forecast. A tool that says "this will work" cannot be checked; one that says "this is live and three weeks old" can. Outcomes are written back and scored.

RUNNING

Weekly capture across seven service lanes, an angle API, and a closed feedback loop into the CRM.

WOULD FIT

Performance agencies and media buyers — the one where an operator with existing clients would see value on the first Monday.

dragontrade-agent · Algom listener in AIPA_AITCF

DragonTrade and Algom

Two smaller systems, included for completeness

DRAGONTRADE 14,323 LOC ALGOM 92 leads

repo: dragontrade-agent

CRM: [CLIENT-ALGOM] · 92

WHY I BUILT IT

DragonTrade was built to learn how to run an agent that must never take an irreversible action without a human — trading education and paper trading only, never live money. **Algom** listens to a public post stream for buying-intent signals and files them as leads; I built it to test whether intent could be detected before someone fills in a form.

DESIGNED

Both follow the same rule as everything else: the agent proposes, a human disposes. Algom writes to the same CRM ledger with its own prefix so its yield is measurable against every other source.

RUNNING

DragonTrade runs with its own dashboard, paper trading only. Algom has filed 92 leads, 30 of them in the last week — the detection works; what sits behind it has never been pointed at a real audience.

WOULD FIT

Included because you asked what is in each repo. If the interest is commercial, the systems above are the ones worth the time.

PART 3

The decisions that repeat

Reading back over fifteen months, the same handful of judgments show up in every system. These are the transferable part — more so than any individual product.

PRINCIPLE	WHERE IT CAME FROM
Never a single point of failure for money	One API balance hitting zero should not stop a business. Five providers, three chains, ordered per use case.
Fail open or fail closed — decide	Alerting fails open: a duplicate notice beats silence. Publishing fails closed: better nothing than something unsourceable.

per system

A human on the last click	Removed auto-apply after it produced volume and no outcomes. Anything that reaches another person waits for one tap.
Measure at the edge, not the source	A commit is not a page, a build is not a deploy, an accepted email is not a delivered one. Verify where the request actually lands.
Refuse to report unmeasured as zero	The citation tracker exits non-zero without a key. A zero that means "nothing counted" looks identical to one that means "nothing happened".
Every record names its author	Ten agents write to one CRM. A prefix makes provenance a filter instead of an investigation.
Write the failure down in public	The AI Ops Wiki. Each entry has a named failure mode and a verified number. It is the cheapest way I know to stop repeating a mistake.

The thing I would want a business to take from this is not any single agent. It is that all of them were built by one person who had to make them survive unattended — which forces a different set of decisions than a team with an on-call rota ever has to make.

PART 4

What this is, and what it is not

THE ONE THING MISSING

What exists: nine systems running unattended on one server, built and maintained alone over fifteen months, with the design reasoning documented as it happened rather than reconstructed afterwards. Every one solves a problem a real business has.

What does not: a route to those businesses. I built supply for fifteen months from a country where I had no network, in a second language, with no marketing budget. The systems have never been placed in front of the people who need them.

That gap is not a quality problem, and it is the only one I cannot close alone. Everything else on this page I built by myself.

WHAT WOULD ACTUALLY HELP

- **An introduction to one business with the matching problem.** A relocation firm or international school for EspaLuz, a performance agency for Atlas, an agency selling SEO for the visibility engine, a team drowning in inbound for the concierge.
- **Or contract engineering work.** If the judgment above reads as useful, the fastest honest route is me building this class of system for someone who already has customers.

Everything here is verifiable. The code is on GitHub, the systems are running now, and every figure in this document was read from production on the day it was written. Happy to walk anyone through any of it live — including the parts that broke.

Elena Revicheva · Aldeazz AI Lab, Panama City · UTC−5

Live systems: [aideazz.xyz/portfolio](#) · Incident record: [aideazz.xyz/ai-ops-wiki.html](#) · Free audit: [aideazz.xyz/api](#)

Figures read from the running systems on 24 August 2026. Nothing in this document came from memory or from an earlier document. Source: [webhook.aideazz.xyz/doc/nine-systems](#)